

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Eval Set Review - __SKILL_NAME_PLACEHOLDER__</title>
  <link rel="preconnect" href="https://fonts.googleapis.com">
  <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
  <link
href="https://fonts.googleapis.com/css2?family=Poppins:wght@500;600&family=Lora:wght@4
00;500&display=swap" rel="stylesheet">
  <style>
    * { box-sizing: border-box; margin: 0; padding: 0; }
    body { font-family: 'Lora', Georgia, serif; background: #faf9f5; padding: 2rem;
color: #141413; }
    h1 { font-family: 'Poppins', sans-serif; margin-bottom: 0.5rem; font-size: 1.5rem;
}
    .description { color: #b0aea5; margin-bottom: 1.5rem; font-style: italic;
max-width: 900px; }
    .controls { margin-bottom: 1rem; display: flex; gap: 0.5rem; }
    .btn { font-family: 'Poppins', sans-serif; padding: 0.5rem 1rem; border: none;
border-radius: 6px; cursor: pointer; font-size: 0.875rem; font-weight: 500; }
    .btn-add { background: #6a9bcc; color: white; }
    .btn-add:hover { background: #5889b8; }
    .btn-export { background: #d97757; color: white; }
    .btn-export:hover { background: #c4613f; }
    table { width: 100%; max-width: 1100px; border-collapse: collapse; background:
white; border-radius: 6px; overflow: hidden; box-shadow: 0 1px 3px rgba(0,0,0,0.08); }
    th { font-family: 'Poppins', sans-serif; background: #141413; color: #faf9f5;
padding: 0.75rem 1rem; text-align: left; font-size: 0.875rem; }
    td { padding: 0.75rem 1rem; border-bottom: 1px solid #e8e6dc; vertical-align: top;
}
    tr:nth-child(even) td { background: #faf9f5; }
    tr:hover td { background: #f3f1ea; }
    .section-header td { background: #e8e6dc; font-family: 'Poppins', sans-serif;
font-weight: 500; font-size: 0.8rem; color: #141413; text-transform: uppercase;
letter-spacing: 0.05em; }
    .query-input { width: 100%; padding: 0.4rem; border: 1px solid #e8e6dc;
border-radius: 4px; font-size: 0.875rem; font-family: 'Lora', Georgia, serif; resize:
vertical; min-height: 60px; }
    .query-input:focus { outline: none; border-color: #d97757; box-shadow: 0 0 0 2px
rgba(217,119,87,0.15); }
    .toggle { position: relative; display: inline-block; width: 44px; height: 24px; }
    .toggle input { opacity: 0; width: 0; height: 0; }
    .toggle .slider { position: absolute; inset: 0; background: #b0aea5; border-radius:
24px; cursor: pointer; transition: 0.2s; }
    .toggle .slider::before { content: ""; position: absolute; width: 18px; height:
18px; left: 3px; bottom: 3px; background: white; border-radius: 50%; transition: 0.2s;
}
    .toggle input:checked + .slider { background: #d97757; }
    .toggle input:checked + .slider::before { transform: translateX(20px); }
    .btn-delete { background: #c44; color: white; padding: 0.3rem 0.6rem; border: none;
border-radius: 4px; cursor: pointer; font-size: 0.75rem; font-family: 'Poppins',
sans-serif; }
    .btn-delete:hover { background: #a33; }

```

```

    .summary { margin-top: 1rem; color: #b0aea5; font-size: 0.875rem; }
  </style>
</head>
<body>
  <h1>Eval Set Review: <span id="skill-name">__SKILL_NAME_PLACEHOLDER__</span></h1>
  <p class="description">Current description: <span
id="skill-desc">__SKILL_DESCRIPTION_PLACEHOLDER__</span></p>

  <div class="controls">
    <button class="btn btn-add" onclick="addRow()">+ Add Query</button>
    <button class="btn btn-export" onclick="exportEvalSet()">Export Eval Set</button>
  </div>

  <table>
    <thead>
      <tr>
        <th style="width:65%">Query</th>
        <th style="width:18%">Should Trigger</th>
        <th style="width:10%">Actions</th>
      </tr>
    </thead>
    <tbody id="eval-body"></tbody>
  </table>

  <p class="summary" id="summary"></p>

  <script>
    const EVAL_DATA = __EVAL_DATA_PLACEHOLDER__;

    let evalItems = [...EVAL_DATA];

    function render() {
      const tbody = document.getElementById('eval-body');
      tbody.innerHTML = '';

      // Sort: should-trigger first, then should-not-trigger
      const sorted = evalItems
        .map((item, origIdx) => ({ ...item, origIdx }))
        .sort((a, b) => (b.should_trigger ? 1 : 0) - (a.should_trigger ? 1 : 0));

      let lastGroup = null;
      sorted.forEach(item => {
        const group = item.should_trigger ? 'trigger' : 'no-trigger';
        if (group !== lastGroup) {
          const headerRow = document.createElement('tr');
          headerRow.className = 'section-header';
          headerRow.innerHTML = `<td colspan="3">${item.should_trigger ? 'Should
Trigger' : 'Should NOT Trigger'}</td>`;
          tbody.appendChild(headerRow);
          lastGroup = group;
        }

        const idx = item.origIdx;
        const tr = document.createElement('tr');
        tr.innerHTML = `

```

```

        <td><textarea class="query-input" onchange="updateQuery(${idx},
this.value)">${escapeHtml(item.query)}</textarea></td>
        <td>
            <label class="toggle">
                <input type="checkbox" ${item.should_trigger ? 'checked' : ''}
onchange="updateTrigger(${idx}, this.checked)">
                <span class="slider"></span>
            </label>
            <span
style="margin-left:8px;font-size:0.8rem;color:#b0aea5">${item.should_trigger ? 'Yes' :
'No'}</span>
        </td>
        <td><button class="btn-delete"
onclick="deleteRow(${idx})">Delete</button></td>
    `;
    tbody.appendChild(tr);
    });
    updateSummary();
}

function escapeHtml(text) {
    const div = document.createElement('div');
    div.textContent = text;
    return div.innerHTML;
}

function updateQuery(idx, value) { evalItems[idx].query = value; updateSummary(); }
function updateTrigger(idx, value) { evalItems[idx].should_trigger = value;
render(); }
function deleteRow(idx) { evalItems.splice(idx, 1); render(); }

function addRow() {
    evalItems.push({ query: '', should_trigger: true });
    render();
    const inputs = document.querySelectorAll('.query-input');
    inputs[inputs.length - 1].focus();
}

function updateSummary() {
    const trigger = evalItems.filter(i => i.should_trigger).length;
    const noTrigger = evalItems.filter(i => !i.should_trigger).length;
    document.getElementById('summary').textContent =
        `${evalItems.length} queries total: ${trigger} should trigger, ${noTrigger}
should not trigger`;
}

function exportEvalSet() {
    const valid = evalItems.filter(i => i.query.trim() !== '');
    const data = valid.map(i => ({ query: i.query.trim(), should_trigger:
i.should_trigger }));
    const blob = new Blob([JSON.stringify(data, null, 2)], { type: 'application/json'
});
    const url = URL.createObjectURL(blob);
    const a = document.createElement('a');
    a.href = url;

```

```
    a.download = 'eval_set.json';
    document.body.appendChild(a);
    a.click();
    document.body.removeChild(a);
    URL.revokeObjectURL(url);
  }

  render();
</script>
</body>
</html>
```